Appendix A: Code for maximum

Here we give the Mathematica code used for investigating the maximum. It generates all c-cyclic graphs with a universal vertex and computes both indices.

```
ClearAll["Global`*"];

(*----------Vertex degrees from edge list----------*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*----------Inverse degree index I(G)----------*)
Iindex[edges_List]:=Module[{deg,n},
n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*----------SDD(G)----------*)
SDD[edges_List]:=Module[{deg,n},
n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]];

(*----------All c-cyclic graphs with a universal vertex----------*)
AllGraphs[n_Integer,c_Integer]:=Module[{others,base,allExtraEdges,extraSubsets},
others=Range[2,n];
base=Table[{1,i},{i,others}];(*star: all connected to 1*)
allExtraEdges=Subsets[others,{2}];(*all possible edges among 2..n*)
extraSubsets=Subsets[allExtraEdges,{c}];(*choose c extra edges*)
Map[Join[base,#]&,extraSubsets]];

(*----------Auxiliary function: degree sequence sorted descending----------*)
DegreeSequence[edges_List,n_Integer]:=Sort[VertexDegrees[edges,n],Greater];

(*----------Conjecture check and results display----------*)
CheckConjecture[c_Integer,nMax_Integer]:=Module[{n,graphs,iVals,sVals,
 maxI,maxS,maxIpos,maxSpos,seqI,seqS,sI},
Print["==================== c = ",c," ===================="];
Do[If[n<c+2,Continue[]];
graphs=AllGraphs[n,c];
If[Length[graphs]==0,Continue[]];
iVals=Iindex/@graphs;
sVals=SDD/@graphs;
maxI=Max[iVals];
maxS=Max[sVals];
maxIpos=First[FirstPosition[iVals,maxI]];
maxSpos=First[FirstPosition[sVals,maxS]];
seqI=DegreeSequence[graphs[[maxIpos]],n];
seqS=DegreeSequence[graphs[[maxSpos]],n];
sI=sVals[[maxIpos]];
Print["n = ",n," (",Length[graphs]," graphs)"];
Print["  max I = ",maxI,"   SDD(max I) = ",sI,"   sequence: ",seqI];
Print["  max SDD = ",maxS,"   sequence: ",seqS];
If[seqI==seqS,
Print["  SAME sequence! Conjecture holds for this pair."],
Print["  DIFFERENT sequences! Conjecture DOES NOT hold for this pair."]
];
```

```
Print[""];,{n,c+2,nMax}]];
(*----------EXECUTION for c=3,4,5,6----------*)
Do[CheckConjecture[c,c+5],{c,3,6}]
```

Appendix B: Code for minimum (full enumeration)

```
ClearAll["Global`*"];

(*----------Vertex degrees from edge list----------*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*----------Inverse degree index I(G)----------*)
Iindex[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*----------SDD(G)----------*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*----------Check connectivity of graph given by edge list----------*)
ConnectedQ[edges_List,n_Integer]:=Module[{adj,visited,queue,v},
adj=Table[{},{n}];
Do[AppendTo[adj[[e[[1]]]],e[[2]]];AppendTo[adj[[e[[2]]]],e[[1]]],{e,edges}];
visited=ConstantArray[False,n];
queue={1};
visited[[1]]=True;
While[queue=!={},v=First[queue];
queue=Rest[queue];
Do[If[!visited[[w]],visited[[w]]=True;AppendTo[queue,w]],{w,adj[[v]]}]];
AllTrue[visited,TrueQ]];

(*----------All connected graphs with n vertices and m edges----------*)
AllConnectedGraphs[n_Integer,m_Integer]:=Module[{allEdges,edgeSets},
allEdges=Subsets[Range[n],{2}];
edgeSets=Subsets[allEdges,{m}];
Select[edgeSets,ConnectedQ[#,n]&]];

(*----------All c-cyclic graphs of order n----------*)
AllCCyclicGraphs[n_Integer,c_Integer]:=AllConnectedGraphs[n,n+c-1];

(*----------Degree sequence sorted descending----------*)
DegreeSequence[edges_List,n_Integer]:=Sort[VertexDegrees[edges,n],Greater];

(*----------Maximum number of subsets we allow----------*)
maxSubsets=500000;

(*----------Main routine for checking minimum----------*)
CheckMinimumConjecture[c_Integer,nMax_Integer]:=
Module[{n,m,totalSubsets,graphs,iVals,sVals,minI,minS,
 minIpos,minSpos,seqI,seqS,sI},
Print["==================== c = ",c," ===================="];
Do[m=n+c-1;
totalSubsets=Binomial[Binomial[n,2],m];
```

```
If[totalSubsets>maxSubsets,
Print["n = ",n," : skipping (",totalSubsets," subsets > ",maxSubsets,")"];
Continue[]];
Print["n = ",n," (",totalSubsets," subsets) ..."];
graphs=AllCCyclicGraphs[n,c];
Print["  number of connected graphs = ",Length[graphs]];
If[Length[graphs]==0,Continue[]];
iVals=Iindex/@graphs;
sVals=SDD/@graphs;
minI=Min[iVals];
minS=Min[sVals];
minIpos=First[FirstPosition[iVals,minI]];
minSpos=First[FirstPosition[sVals,minS]];
seqI=DegreeSequence[graphs[[minIpos]],n];
seqS=DegreeSequence[graphs[[minSpos]],n];
sI=sVals[[minIpos]];
Print["  min I = ",minI,"   SDD(min I) = ",sI,"   sequence: ",seqI];
Print["  min SDD = ",minS,"   sequence: ",seqS];
If[sI==minS,
Print["  \[Checkmark] Same SDD values \[Dash] minima coincide."],
Print["  \[Cross] DIFFERENT SDD values \[Dash] COUNTEREXAMPLE!"]];
If[seqI==seqS,
Print["  \[Checkmark] Same degree sequence \[Dash] conjecture holds."],
Print["  \[Cross] DIFFERENT degree sequences \[Dash] conjecture DOES NOT HOLD."]];
Print[""];,{n,c+2,nMax}]];

(*----------EXECUTION for c=3,4,5,6----------*)
Do[CheckMinimumConjecture[c,7],{c,3,6}]
```

Appendix C: Code for minimum for $c = 6, n = 8$ (search by degree sequences)

```
ClearAll["Global`*"];

(*----------Vertex degrees from edge list----------*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*----------Inverse degree index I(G)----------*)
Iindex[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*----------SDD(G)----------*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*----------Check connectivity (BFS)----------*)
ConnectedQ[edges_List,n_Integer]:=Module[{adj,visited,queue,v},
adj=Table[{},{n}];
Do[AppendTo[adj[[e[[1]]]],e[[2]]];
AppendTo[adj[[e[[2]]]],e[[1]]],{e,edges}];
visited=ConstantArray[False,n];
queue={1};
```

```
visited[[1]]=True;
While[queue=!={},v=First[queue];queue=Rest[queue];
Do[If[!visited[[w]],visited[[w]]=True;AppendTo[queue,w]],{w,adj[[v]]}]];
AllTrue[visited,TrueQ]];

(*----------Backtracking: all realizations of a degree sequence----------*)
AllGraphsFromDegreeSequence[degseq_List]:=
Module[{n=Length[degseq],results={}},
ClearAll[backtrack];
backtrack[deg_,edges_]:=Module[{i,d,others,subset,newDegs,newEdges},
If[AllTrue[deg,#==0&],
AppendTo[results,Sort[edges]];
Return[]
];
i=First[Ordering[deg,-1]];
d=deg[[i]];
others=Select[Range[n],#!=i&&deg[[#]]>0&];
If[Length[others]<d,Return[]];
Do[subset=Sort[s];
newDegs=deg;
newDegs[[i]]=0;
newEdges=edges;
Do[newDegs[[v]]=newDegs[[v]]-1;
AppendTo[newEdges,{i,v}],{v,subset}];
If[Min[newDegs]>=0,backtrack[newDegs,newEdges]],
{s,Subsets[others,{d}]}]
];
backtrack[degseq,{}];
results
];

(*----------MAIN PART----------*)
n=8;
c=6;
m=n+c-1;(*13*)

(*All possible degree sequences for n=8,m=13*)
allSeq=IntegerPartitions[2*m,{n},Range[1,n-1]];
(*already nonincreasing, elements 1..7*)
Print["Number of candidate degree sequences: ",Length[allSeq]];

results={};
Do[
seq=seqcand;
graphs=AllGraphsFromDegreeSequence[seq];
conn=Select[graphs,ConnectedQ[#,n]&];
If[Length[conn]>0,
sddVals=SDD/@conn;
minSDD=Min[sddVals];
AppendTo[results,{seq,minSDD,Length[conn]}];
Print[seq," -> min SDD = ",minSDD," (",Length[conn]," graphs)"];
],
{seqcand,allSeq}
];

(*Smallest SDD*)
If[results=!={},
sorted=SortBy[results,#[[2]]&];
Print["\n=== RESULTS ==="];
Print["Global minimum SDD: ",sorted[[1,2]]," for sequence ",sorted[[1,1]]];
Print["Smallest SDD among minimal I-sequence (4,4,3,3,3,3,3,3) is 53/2 = ",53/2];
```

```
Print["All sequences and their minSDD:"];
TableForm[sorted,TableHeadings->{None,{"sequence","min SDD","number of graphs"}}]
,
Print["No sequence has a connected realization! (Impossible)"]
]
```

Appendix D: Code for minimum for $c = 6, n = 9$ (search by degree sequences)

```
ClearAll["Global`*"];

(*----------Vertex degrees from edge list----------*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*----------Inverse degree index I(G)----------*)
Iindex[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*----------SDD(G)----------*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*----------Check connectivity (BFS)----------*)
ConnectedQ[edges_List,n_Integer]:=Module[{adj,visited,queue,v},
adj=Table[{},{n}];
Do[AppendTo[adj[[e[[1]]]],e[[2]]];
AppendTo[adj[[e[[2]]]],e[[1]]],{e,edges}];
visited=ConstantArray[False,n];
queue={1};
visited[[1]]=True;
While[queue=!={},v=First[queue];queue=Rest[queue];
Do[If[!visited[[w]],visited[[w]]=True;AppendTo[queue,w]],{w,adj[[v]]}]];
AllTrue[visited,TrueQ]];

(*----------Backtracking: all realizations of a degree sequence----------*)
AllGraphsFromDegreeSequence[degseq_List]:=
Module[{n=Length[degseq],results={}},
ClearAll[backtrack];
backtrack[deg_,edges_]:=Module[{i,d,others,subset,newDegs,newEdges},
If[AllTrue[deg,#==0&],
AppendTo[results,Sort[edges]];
Return[]
];
i=First[Ordering[deg,-1]];
d=deg[[i]];
others=Select[Range[n],#!=i&&deg[[#]]>0&];
If[Length[others]<d,Return[]];
Do[subset=Sort[s];
newDegs=deg;
newDegs[[i]]=0;
newEdges=edges;
Do[newDegs[[v]]=newDegs[[v]]-1;
```

```
AppendTo[newEdges,{i,v}],{v,subset}];
If[Min[newDegs]>=0,backtrack[newDegs,newEdges]],
{s,Subsets[others,{d}]}]
];
backtrack[degseq,{}];
results
];

(*----------MAIN PART----------*)
n=9;
c=6;
m=n+c-1;(*14*)
(*All possible degree sequences for n=9,m=14*)
allSeq=IntegerPartitions[2*m,{n},Range[1,n-1]];
(*already nonincreasing, elements 1..8*)
Print["Number of candidate degree sequences: ",Length[allSeq]];

results={};
Do[
seq=seqcand;
graphs=AllGraphsFromDegreeSequence[seq];
conn=Select[graphs,ConnectedQ[#,n]&];
If[Length[conn]>0,
sddVals=SDD/@conn;
minSDD=Min[sddVals];
AppendTo[results,{seq,minSDD,Length[conn]}];
Print[seq," -> min SDD = ",minSDD," (",Length[conn]," graphs)"];
],
{seqcand,allSeq}
];

(*Smallest SDD*)
If[results=!={},
sorted=SortBy[results,#[[2]]&];
Print["\n=== RESULTS ==="];
Print["Global minimum SDD: ",sorted[[1,2]]," for sequence ",sorted[[1,1]]];
Print["Smallest SDD among minimal I-sequence (4,3,3,3,3,3,3,3,3) is ",
SDD[AllGraphsFromDegreeSequence[{4,3,3,3,3,3,3,3,3}][[1]]]];
Print["All sequences and their minSDD:"];
TableForm[sorted,TableHeadings->{None,{"sequence","min SDD","number of graphs"}}]
,
Print["No sequence has a connected realization! (Impossible)"]
]
```